

MINI PROJET TRAITEMENT IMAGE

Informatique 3 (API2)

Année universitaire : 2025 – 2026

—

Rapport technique – Implémentation
MATLAB

Réalisé par :

AMBDLOUFATTAH MOHAMED HALIM

(N°20)

AKEBBAD MOHCINE (N°18)

AMDINI AYOUB (N°21)

AXES :

- Introduction
- Partie I : Les opérations d'entrée/sortie sur les images
- Partie II : Les images Noir et blanc
- Partie III : Les images en niveau de gris
- Partie IV : Opérations élémentaires sur les images en mode gris

-Partie VI : Les images RGB

- Implémentation d'un menu interactif pour l'amélioration du mini-projet (supplémentaire)
- conclusion

Introduction :

Ce rapport présente la réalisation d'un programme de traitement d'images sous MATLAB.

L'objectif principal de ce travail est de manipuler des images numériques à l'aide d'opérations fondamentales telles que la lecture d'images, la création d'images binaires, l'inversion des niveaux de gris, les symétries, le calcul de la luminance et du contraste, ainsi que la manipulation d'images RGB et leur conversion en niveaux de gris.

Le programme est structuré autour d'un menu interactif permettant à l'utilisateur de choisir l'opération souhaitée.

-Partie I : Les opérations d'entrée/sortie sur les images

Fonction `lectureImage`: la fonction `lectureImage` a pour rôle de lire une image à partir de son chemin d'accès et de la préparer pour les traitements ultérieurs. Elle permet également de gérer les erreurs de lecture et d'uniformiser le format des images utilisées dans le programme.

Dans un premier temps, la fonction reçoit en paramètre `filename`, qui représente le chemin ou le nom du fichier image à charger. La lecture de l'image est effectuée à l'aide de la fonction intégrée `imread`, qui retourne une matrice représentant l'image. En MATLAB, une image est stockée sous forme matricielle : une image en niveaux de gris est représentée par une matrice bidimensionnelle, tandis qu'une image en couleur est représentée par une matrice tridimensionnelle correspondant aux trois composantes Rouge, Vert et Bleu (RGB).

La fonction vérifie ensuite si l'image est en couleur en testant la taille de la troisième dimension de la matrice. Si cette dimension est égale à 3, cela signifie que l'image est une image RGB. Dans ce cas, l'image est convertie en niveaux de gris à l'aide de la fonction `rgb2gray`. Cette conversion permet de réduire l'information de couleur à une seule intensité lumineuse par pixel, ce qui simplifie

les traitements ultérieurs tels que le calcul du négatif, de la luminance ou du contraste.

Enfin, l'ensemble du code est placé à l'intérieur d'un bloc try-catch. Ce mécanisme permet de gérer les erreurs éventuelles, notamment lorsque le fichier image n'existe pas ou que le chemin fourni est invalide. En cas d'erreur, un message explicite est affiché afin d'informer l'utilisateur du problème, ce qui améliore la robustesse du programme.

```
% Fonction pour lire une image et la convertir en niveaux de gris
function Img = lectureImage(filename)
    try
        Img = imread(filename);
        if size(Img, 3) == 3
            Img = rgb2gray(Img);
        end
    catch
        error('Image non trouvée ou chemin invalide');
    end
end
```

Fonction afficher_image (affichage d'une image)

La fonction afficher_image permet de **visualiser une image** en utilisant l'interface graphique de MATLAB. Elle prend en paramètre Img, qui peut être une image en niveaux de gris ou une image couleur (RGB).

Paramètre d'entrée

- Img : matrice représentant l'image.
 - Si Img est tridimensionnelle avec trois canaux, il s'agit d'une image RGB.

- Sinon, il s'agit d'une image en niveaux de gris.

Description du traitement

1. La fonction crée une **nouvelle figure** MATLAB pour afficher l'image à l'aide de la commande figure;.
2. Elle teste ensuite si l'image est une image couleur :
 - `ndims(Img) == 3` vérifie que l'image possède trois dimensions.
 - `size(Img, 3) == 3` vérifie que la troisième dimension correspond aux trois canaux R, G et B.
3. Selon le type d'image :
 - **Image RGB** : la fonction `imshow(Img)` est utilisée pour afficher directement l'image couleur.
 - **Image en niveaux de gris** : `imshow(Img, [])` affiche l'image en ajustant automatiquement la plage de valeurs des pixels. La commande `colormap(gray)` applique la palette de gris pour que les intensités soient correctement représentées.
4. La fonction ajoute un **titre à la figure** avec `title("Image blanche_noir")`.
5. La commande `axis off` supprime l'affichage des axes pour un rendu plus esthétique, centré uniquement sur l'image.

Résultat

- Une fenêtre MATLAB affiche l'image correctement visualisée, en couleur ou en niveaux de gris.
- Cette fonction est essentielle pour **vérifier visuellement les résultats des traitements d'images**, comme l'inversion, la symétrie ou la conversion en niveaux de gris.

```
% affichage
function afficher_image(Img)
    figure;
    if ndims(Img) == 3 && size(Img, 3) == 3
        imshow(Img);
    else
        imshow(Img, []);
        colormap(gray);
    end
    title("Image blanche_noir");
    axis off;
end
```

-Partie II : Les images Noir et blanc :

Fonction image_noir_blanche:

La fonction `image_noir_blanche` permet de générer une image binaire de type *damier* à partir des dimensions fournies par l'utilisateur. Elle est utilisée pour illustrer la création artificielle d'une image numérique et la manipulation directe des pixels à l'aide des indices de matrice.

La fonction prend en paramètres `h` et `l`, qui représentent respectivement la hauteur (nombre de lignes) et la largeur (nombre de colonnes) de l'image à créer. Une matrice de taille $h \times l$ est d'abord initialisée à l'aide de la fonction `ones`, ce qui permet de réserver l'espace mémoire nécessaire pour l'image.

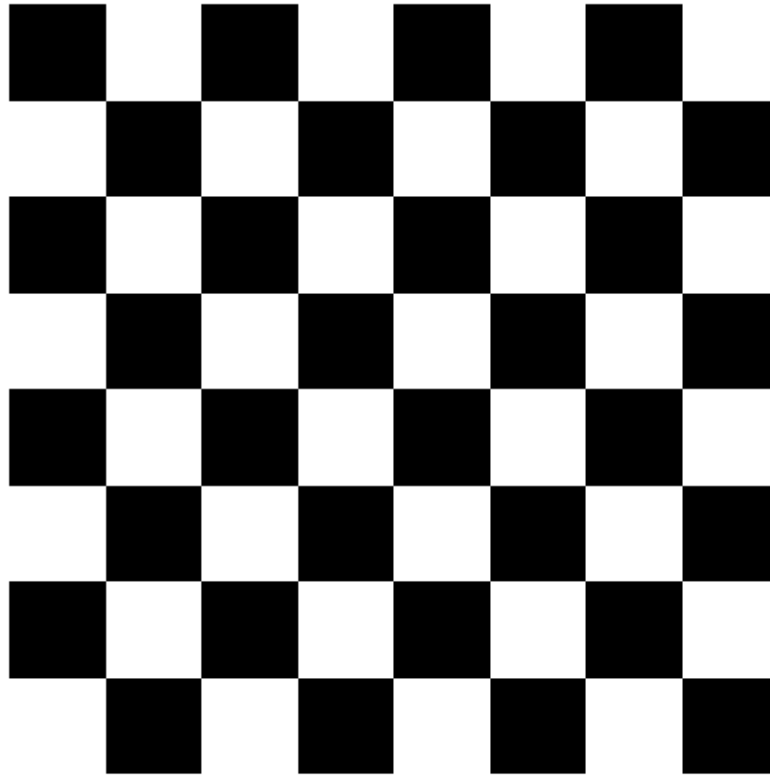
Ensuite, deux boucles imbriquées parcourent l'ensemble des pixels de l'image. Pour chaque position (i, j) , la valeur du pixel est calculée à l'aide de l'opération modulo appliquée à la somme des indices de ligne et de colonne. Lorsque $i + j$ est pair, la valeur retournée par `mod(i + j, 2)` est égale à 0, correspondant à un pixel noir. En revanche, lorsque $i + j$ est impair, la valeur obtenue est égale à 1, correspondant à un pixel blanc.

Ce mécanisme permet d'alterner automatiquement les pixels noirs et blancs, créant ainsi un motif régulier en forme de damier. L'image générée est une image binaire, où

chaque pixel ne peut prendre que deux valeurs possibles (0 ou 1), ce qui constitue une base simple mais efficace pour comprendre la représentation matricielle des images et les principes fondamentaux du traitement d'images numériques.

```
% définir l'image comme damier
function Img = image_noir_blanche(h, l)
    Img = ones(h, l);
    for i = 1:h
        for j = 1:l
            Img(i, j) = mod(i + j, 2);
        end
    end
end
```

Image blanche_noir



La fonction negatif :

La fonction negatif permet de calculer le négatif d'une image en inversant les niveaux d'intensité de ses pixels. Cette transformation est couramment utilisée en traitement d'images afin de mettre en évidence certaines structures ou zones de contraste.

La fonction prend en paramètre *Img*, qui représente une image en niveaux de gris stockée sous forme de matrice. Avant d'effectuer l'inversion, une étape de normalisation est réalisée. En effet, les valeurs des pixels peuvent être codées soit dans l'intervalle $[0, 1]$, soit dans l'intervalle $[0, 255]$. La condition $\max(\text{Img}(:)) > 1$ permet de vérifier si l'image est codée sur 8 bits. Si tel est le cas, l'image est convertie en type double puis divisée par 255 afin de ramener les intensités dans l'intervalle $[0, 1]$.

Une fois l'image normalisée, le négatif est calculé en appliquant la relation suivante à chaque pixel :

$$N(x, y) = 1 - I(x, y)$$

Cette opération inverse les zones claires et sombres de l'image : les pixels clairs deviennent sombres et inversement. Le résultat est stocké dans la variable result, qui correspond à l'image négative finale.

Cette fonction illustre l'importance de la normalisation des intensités dans le traitement d'images et montre comment une transformation simple peut modifier significativement l'apparence visuelle d'une image.

```
% inverser l'image
function result = negatif(Img)
    % Normaliser entre 0 et 1 si nécessaire
    if max(Img(:)) > 1
        Img = double(Img) / 255;
    end
    result = 1 - Img;
end
```



Partie III. Les images en niveau de gris

Fonction luminance:

La fonction luminance permet de calculer la luminance moyenne d'une image en niveaux de gris. La luminance représente la luminosité globale de l'image et correspond à la moyenne des intensités de tous ses pixels.

La fonction prend en paramètre `Img`, qui est une image en niveaux de gris représentée sous forme d'une matrice de taille

$h \times l$, où

h correspond au nombre de lignes et

l au nombre de colonnes de l'image. Les dimensions de l'image sont obtenues à l'aide de la fonction `size`.

Une variable `somme` est initialisée à zéro afin d'accumuler les valeurs des intensités de tous les pixels. Deux boucles imbriquées parcourent ensuite

l'ensemble de l'image pixel par pixel. À chaque position

(i,j)

(i,j) , la valeur du pixel est convertie en type double afin d'assurer une précision correcte lors des calculs, puis ajoutée à la somme totale.

Une fois le parcours de l'image terminé, la luminance moyenne est calculée en divisant la somme des intensités par le nombre total de pixels, soit $h \times l$. Le résultat obtenu correspond à la valeur moyenne de luminosité de l'image et est retourné par la fonction dans la variable moyenne.

Cette mesure est particulièrement utile pour analyser la clarté globale d'une image et constitue une base importante pour d'autres traitements tels que le calcul du contraste ou l'amélioration de la qualité visuelle.

```

% fonction de luminance
function moyenne = luminance(Img)
    [h, l] = size(Img);
    somme = 0;
    for i = 1:h
        for j = 1:l
            somme = somme + double(Img(i, j));
        end
    end
    moyenne = somme / (h * l);
end

```

Fonction contrast:

La fonction `contrast` permet de calculer le **contraste d'une image en niveaux de gris**. Le contraste mesure la dispersion des valeurs d'intensité autour de la luminance moyenne et donne une idée de la variation de luminosité à travers l'image. Un contraste élevé signifie que l'image contient à la fois des zones très claires et très sombres, tandis qu'un contraste faible indique que les intensités sont proches de la moyenne.

La fonction prend en paramètre `Img`, qui représente l'image sous forme d'une matrice de taille $h \times l$, où

h est le nombre de lignes et l le nombre de colonnes. La luminance moyenne de l'image est d'abord calculée en appelant la fonction `luminance(lmg)` et stockée dans la variable `lum`.

Une variable `s` est initialisée à zéro pour accumuler la somme des écarts au carré des intensités par rapport à la luminance. Deux boucles imbriquées parcourent chaque pixel de l'image. Pour chaque pixel (i, j) , la différence entre la valeur du pixel et la luminance moyenne est élevée au carré, puis ajoutée à la somme totale. Cette opération correspond au calcul de la **variance des intensités**, qui est une mesure statistique du contraste.

Enfin, le contraste est obtenu en divisant cette somme par le nombre total de pixels $h \times l$. La valeur retournée par la fonction dans la variable `c` représente le **contraste global de l'image** et peut être utilisée pour analyser la qualité visuelle ou pour ajuster des traitements d'amélioration d'image.

```

% Retourne le contraste d'une image
function c = contrast(Img)
    [h, l] = size(Img);
    lum = luminance(Img);

    s = 0;
    for i = 1:h
        for j = 1:l
            s = s + (double(Img(i, j)) - lum) ^ 2;
        end
    end
    c = s / (h * l);
end

```

Fonction profondeur:

La fonction profondeur permet de déterminer la **valeur maximale d'intensité** parmi tous les pixels d'une image en niveaux de gris. Cette valeur correspond à la **profondeur lumineuse maximale** de l'image et peut être utilisée pour analyser l'échelle de gris ou adapter certains traitements comme la normalisation ou le contraste.

La fonction prend en paramètre *Img*, qui représente l'image sous forme d'une matrice de dimensions $h \times l$, où h est le nombre de lignes et l le nombre de colonnes.

Pour commencer, une variable `maxi` est initialisée avec la valeur du premier pixel de l'image `Img(1,1)`, convertie en type `double` pour assurer la précision des calculs. Ensuite, deux boucles imbriquées parcourent chaque pixel de la matrice. À chaque position (i, j) , la valeur du pixel est comparée à la valeur actuelle de `maxi`. Si le pixel courant est plus lumineux que la valeur stockée, `maxi` est mise à jour avec cette nouvelle valeur.

À la fin du parcours complet de l'image, `maxi` contient la **valeur maximale d'un pixel** de l'image, qui est ensuite retournée par la fonction.

Cette opération est simple mais essentielle pour des traitements tels que l'échelle dynamique, le contraste adaptatif et l'analyse statistique des images.

```

% Retourne la valeur maximale d'un pixel dans l'image Img
function maxi = profondeur(Img)
    maxi = double(Img(1, 1));
    [h, l] = size(Img);
    for i = 1:h
        for j = 1:l
            if double(Img(i, j)) > maxi
                maxi = double(Img(i, j));
            end
        end
    end
end

function img = Ouvrir(A)
    img = lectureImage(A);
end

function result = inverser(img)
    p = 255;
    result = p - double(img);
end

```

Fonction Ouvrir:

La fonction Ouvrir permet de **charger l'image** notée A dans l'énoncé du projet. Dans le programme, cette image correspond à un fichier déterminé, mais l'utilisateur peut **fournir le chemin d'accès local** de l'image A pour l'importer.

La fonction utilise `lectureImage(A)` pour lire l'image et la convertir en niveaux de gris si nécessaire. Elle

retourne l'image sous forme de matrice, prête à être utilisée pour les traitements suivants.

Cette approche permet de **centraliser la lecture de l'image** et de simplifier le code principal, tout en donnant la flexibilité d'utiliser des images différentes fournies par l'utilisateur.

```
function img = Ouvrir(A)
    img = lectureImage(A);
end
```

Partie IV. Opérations élémentaires sur les images en mode gris :

Fonction inverser:

La fonction inverser permet de calculer le **négatif d'une image** codée sur 8 bits, c'est-à-dire une image dont les valeurs des pixels sont comprises entre 0 et 255. L'objectif de cette transformation est d'inverser les zones claires et sombres de l'image, ce qui peut aider à mettre en évidence certains détails visuels.

Paramètre d'entrée

- `img` : matrice représentant l'image en niveaux de gris. Chaque élément de la matrice correspond à l'intensité lumineuse d'un pixel.

Description du traitement

1. Une constante `p` est définie avec la valeur 255, correspondant à l'intensité maximale possible pour un pixel sur 8 bits.
2. La fonction effectue l'opération suivante sur chaque pixel de l'image :

$$\text{result}(x, y) = 255 - \text{img}(x, y)$$

Cette soustraction inverse les valeurs de luminosité :

- Les pixels sombres deviennent clairs.
 - Les pixels clairs deviennent sombres.
3. La conversion `double(img)` est utilisée pour assurer que les calculs sont précis et éviter les erreurs de type lors de la soustraction.

Résultat

- **result** : matrice représentant l'image négative, prête à être affichée ou traitée par d'autres fonctions (par exemple pour l'analyse du contraste ou de la luminance).

La fonction **inverser** permet de calculer le **négatif d'une image** codée sur 8 bits, c'est-à-dire une image dont les valeurs des pixels sont comprises entre 0 et 255. L'objectif de cette transformation est d'inverser les zones claires et sombres de l'image, ce qui peut aider à mettre en évidence certains détails visuels.

Paramètre d'entrée

- **img** : matrice représentant l'image en niveaux de gris. Chaque élément de la matrice correspond à l'intensité lumineuse d'un pixel.

Description du traitement

1. Une constante **p** est définie avec la valeur 255, correspondant à l'intensité maximale possible pour un pixel sur 8 bits.
2. La fonction effectue l'opération suivante sur chaque pixel de l'image :

$$\text{result}(x, y) = 255 - \text{img}(x, y)$$

Cette soustraction inverse les valeurs de luminosité :

- Les pixels sombres deviennent clairs.
 - Les pixels clairs deviennent sombres.
3. La conversion `double(img)` est utilisée pour assurer que les calculs sont précis et éviter les erreurs de type lors de la soustraction.

Résultat

- `result` : matrice représentant l'image négative, prête à être affichée ou traitée par d'autres fonctions (par exemple pour l'analyse du contraste ou de la luminance).

```
function result = inverser(img)
    p = 255;
    result = p - double(img);
end
```



Fonction flipH (symétrie verticale):

La fonction flipH permet de réaliser la **symétrie verticale d'une image**, c'est-à-dire de retourner l'image de gauche à droite autour de son axe vertical central. Cette transformation est utile pour créer des effets miroir ou pour l'analyse comparative d'images.

Paramètre d'entrée

- img : matrice représentant l'image en niveaux de gris. Chaque élément correspond à l'intensité d'un pixel.

Description du traitement

1. Les dimensions de l'image sont récupérées à l'aide de $[L, C] = \text{size}(\text{img})$, où L est le nombre de lignes et C le nombre de colonnes.
2. La matrice est convertie en double pour assurer la précision des manipulations.
3. Deux boucles imbriquées parcourent chaque pixel de l'image :
 - La boucle extérieure parcourt chaque ligne i .
 - La boucle intérieure parcourt les colonnes jusqu'au milieu de l'image $\text{floor}(C/2)$.
4. À chaque itération, le pixel situé à la colonne j est échangé avec celui situé à la colonne symétrique par rapport au centre, $C - j + 1$.
 - Cette opération inverse la position des pixels sur l'axe horizontal.
 - Les pixels de gauche passent à droite et ceux de droite passent à gauche, tout en conservant les valeurs d'intensité.

Résultat

- result : matrice représentant l'image retournée horizontalement, correspondant à la symétrie verticale de l'image originale.

Cette fonction illustre l'utilisation de **manipulations directes de matrices pour transformer des images** et constitue une opération de base en traitement d'images numériques.

```
% Symétrie verticale (gauche-droite)
function result = flipH(img)
    [L, C] = size(img);
    img_double = double(img);

    for i = 1:L
        for j = 1:floor(C / 2)
            t = img_double(i, j);
            img_double(i, j) = img_double(i, C - j + 1);
            img_double(i, C - j + 1) = t;
        end
    end

    result = img_double;
end
```



Fonction poserV (assemblage vertical d'images)

La fonction `poserV` permet de **superposer deux images verticalement**, c'est-à-dire de placer une image au-dessus de l'autre, pour créer une image unique combinée. Cette opération est particulièrement utile pour comparer visuellement deux images ou créer des compositions.

Paramètres d'entrée

- `img1` : première image (matrice de pixels).
- `img2` : deuxième image (matrice de pixels).

Description du traitement

1. Les dimensions des deux images sont récupérées : $[L1, C1] = \text{size}(\text{img1})$ et $[L2, C2] = \text{size}(\text{img2})$.
2. La fonction vérifie que les **largeurs des deux images sont identiques** ($C1 == C2$). Si elles diffèrent, la fonction retourne une matrice vide, car l'assemblage vertical n'est pas possible.
3. Une matrice resultat est initialisée avec des dimensions $(L1 + L2, C1)$ pour contenir les deux images.
4. Les pixels de la première image sont copiés dans les premières lignes de resultat.
5. Les pixels de la deuxième image sont copiés dans les lignes suivantes.

Résultat

- resultat : matrice représentant les deux images empilées verticalement.
- Si les images n'ont pas la même largeur, la fonction retourne une matrice vide et un message d'erreur peut être affiché.

Cette fonction permet de **combiner deux images en hauteur**, ce qui est utile pour la visualisation ou le traitement simultané d'images comparables.



Fonction poserH (assemblage horizontal d'images) :

La fonction poserH permet de **placer deux images côte à côte**, c'est-à-dire de créer une nouvelle image

en juxtaposant l'une à droite de l'autre. Elle est utilisée pour créer des compositions horizontales ou comparer deux images sur le même plan.

Paramètres d'entrée

- `img1` : première image (matrice de pixels).
- `img2` : deuxième image (matrice de pixels).

Description du traitement

1. Les dimensions des deux images sont récupérées : $[L1, C1] = \text{size}(\text{img1})$ et $[L2, C2] = \text{size}(\text{img2})$.
2. La fonction vérifie que les **hauteurs des deux images sont identiques** ($L1 == L2$). Si elles diffèrent, la fonction retourne une matrice vide, car l'assemblage horizontal n'est pas possible.
3. Une matrice `resultat` est initialisée avec des dimensions $(L1, C1 + C2)$ pour contenir les deux images.
4. Les pixels de la première image sont copiés dans les premières colonnes de `resultat`.

5. Les pixels de la deuxième image sont copiés dans les colonnes suivantes.

Résultat

- resultat : matrice représentant les deux images juxtaposées horizontalement.
- Si les images n'ont pas la même hauteur, la fonction retourne une matrice vide et un message d'erreur peut être affiché.

Ces fonctions illustrent l'importance de **la manipulation directe des matrices pour combiner des images**, une technique fondamentale dans le traitement et la visualisation d'images numériques.

```

% Poser horizontalement img2 à droite de img1
function resultat = poserH(img1, img2)
    [L1, C1] = size(img1);
    [L2, C2] = size(img2);

    if L1 ~= L2
        resultat = [];
        return;
    end

    resultat = zeros(L1, C1 + C2);

    for i = 1:L1
        for j = 1:C1
            resultat(i, j) = double(img1(i, j));
        end

        for j = 1:C2
            resultat(i, j + C1) = double(img2(i, j));
        end
    end
end

function imageRGB = initImageRGB(n, m)

```



Partie VI : Les images RGB :

Fonction `initImageRGB` (initialisation d'une image RGB aléatoire) :

La fonction `initImageRGB` permet de **créer une image couleur aléatoire** de dimensions $n \times m$, où chaque pixel possède des valeurs aléatoires pour les trois composantes Rouge (R), Vert (G) et Bleu (B). Cette fonction est utile pour tester des algorithmes de traitement d'images sans avoir besoin de fichiers externes.

Paramètres d'entrée

- n : nombre de lignes de l'image (hauteur).
- m : nombre de colonnes de l'image (largeur).

Description du traitement

1. Une matrice `imageRGB` de dimensions $3 \times n \times m$ est initialisée avec des zéros.

- La première dimension représente les **canaux de couleur** :
 - $1 \rightarrow \text{Rouge (R)}$

- 2 → Vert (G)

- 3 → Bleu (B)

- Les deux autres dimensions correspondent aux lignes et colonnes de l'image.

2. Trois boucles imbriquées parcourent chaque canal et chaque pixel :

- La boucle extérieure c parcourt les trois canaux (R, G, B).
- Les boucles i et j parcourent respectivement les lignes et colonnes de l'image.

3. Pour chaque pixel (i,j) et chaque canal c, la valeur est choisie aléatoirement entre 0 et 255 à l'aide de la fonction `randi([0, 255])`.

- Cela simule la couleur d'un pixel dans un espace RGB sur 8 bits.

Résultat

- `imageRGB` : matrice tridimensionnelle représentant une **image couleur aléatoire**.

- Chaque pixel possède trois valeurs correspondant aux intensités R, G et B, permettant de visualiser ou tester des traitements couleur.

Cette fonction illustre l'importance de **la manipulation directe des matrices 3D** pour générer et représenter des images RGB en MATLAB, ce qui est fondamental pour le traitement d'images couleur.

```
function imageRGB = initImageRGB(n, m)
% MATLAB: rand génère entre 0 et 1, on multiplie par 255
imageRGB = zeros(3, n, m);
for c = 1:3
    for i = 1:n
        for j = 1:m
            imageRGB(c, i, j) = randi([0, 255]);
        end
    end
end
end
```



Fonction `symetrie_horizontale` (symétrie haut-bas):

La fonction `symetrie_horizontale` réalise une **symétrie horizontale** d'une image, c'est-à-dire qu'elle retourne l'image **autour de son axe horizontal central**.

Concrètement, les lignes du haut deviennent les lignes du bas et vice-versa, comme si l'image était retournée verticalement.

Paramètre d'entrée

- `img` : matrice représentant l'image en niveaux de gris.

Description du traitement

- La fonction utilise la fonction MATLAB intégrée `flipud` (flip up-down), qui inverse l'ordre des lignes de la matrice.
- Chaque ligne i de l'image originale devient la ligne $L-i+1$ dans l'image résultante, où L est le nombre total de lignes.

Résultat

- `result` : matrice représentant l'image retournée verticalement.
- Cette opération est utilisée pour créer un effet miroir vertical ou pour comparer les parties supérieures et inférieures d'une image.

```
% symétrie horizontale (haut-bas)
function result = symetrie_horizontale(img)
    result = flipud(img);
end
```



Fonction `symetrie_verticale` (symétrie gauche-droite) :

La fonction `symetrie_verticale` réalise une **symétrie verticale** d'une image, c'est-à-dire qu'elle retourne l'image **autour de son axe vertical central**. Les colonnes de gauche deviennent les colonnes de droite et inversement, comme un effet miroir horizontal.

Paramètre d'entrée

- `img` : matrice représentant l'image en niveaux de gris.

Description du traitement

- La fonction utilise la fonction MATLAB intégrée `fliplr` (flip left-right), qui inverse l'ordre des colonnes de la matrice.
- Chaque colonne j de l'image originale devient la colonne $C-j+1$ dans l'image résultante, où C est le nombre total de colonnes.

Résultat

- `result` : matrice représentant l'image retournée horizontalement.
- Cette opération est utile pour créer des effets miroir ou pour des analyses comparatives d'images.



Fonction grayscale (conversion d'une image RGB en niveaux de gris) :

La fonction grayscale permet de **convertir une image couleur (RGB) en image en niveaux de gris**, en réduisant les trois composantes de couleur (Rouge, Vert, Bleu) à une seule intensité lumineuse par pixel. Cette opération est fondamentale dans le traitement d'images, car elle simplifie l'analyse tout en conservant l'information de luminosité.

Paramètre d'entrée

- imageRGB : matrice tridimensionnelle représentant l'image couleur.
 - La première dimension correspond aux canaux de couleur (1 → Rouge, 2 → Vert, 3 → Bleu).
 - Les deuxième et troisième dimensions correspondent aux lignes et colonnes de l'image.

Description du traitement

1. La fonction récupère les dimensions de l'image :

- $m = \text{size}(\text{imageRGB}, 2) \rightarrow$ nombre de lignes.
- $n = \text{size}(\text{imageRGB}, 3) \rightarrow$ nombre de colonnes.

2. Une matrice gris de taille $m \times n$ est initialisée pour stocker l'image en niveaux de gris.

3. Deux boucles imbriquées parcourent chaque pixel de l'image :

- Pour chaque position (i, j) , les valeurs des trois canaux R, G et B sont extraites.
- La valeur du pixel en niveaux de gris est calculée selon la formule :

$$\text{gris}(i, j) = \frac{\max(R, G, B) + \min(R, G, B)}{2}$$

- Cette méthode, appelée **méthode du “gris moyen par extrêmes”**, consiste à prendre la moyenne entre la valeur maximale et la valeur minimale des trois composantes, ce qui permet de conserver la luminosité globale tout en supprimant la couleur.

- La fonction floor est utilisée pour arrondir le résultat à l'entier inférieur.

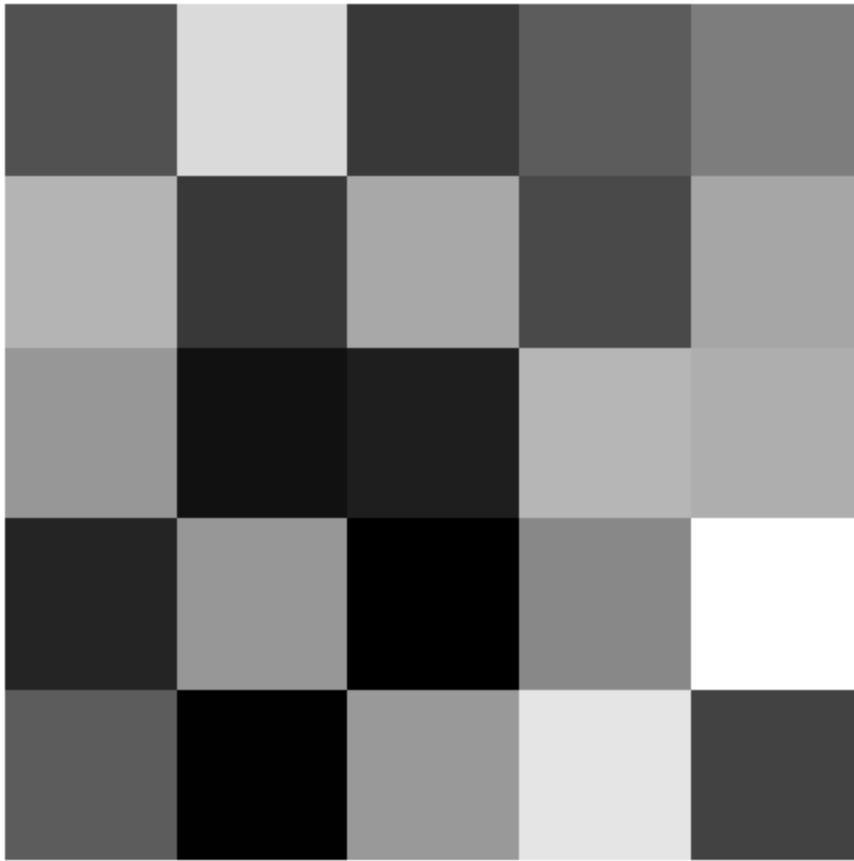
Résultat

- gris : matrice représentant l'image convertie en niveaux de gris, prête à être affichée ou utilisée pour d'autres traitements, comme le calcul de luminance, de contraste ou la transformation en négatif.

Cette fonction illustre l'importance de **la réduction des dimensions de données** pour simplifier le traitement d'images couleur et obtenir des informations de luminosité exploitables.

```
function gris = grayscale(imageRGB)
    m = size(imageRGB, 2);
    n = size(imageRGB, 3);
    gris = zeros(m, n);

    for i = 1:m
        for j = 1:n
            R = imageRGB(1, i, j);
            G = imageRGB(2, i, j);
            B = imageRGB(3, i, j);
            gris(i, j) = floor((max([R, G, B]) + min([R, G, B])) / 2);
        end
    end
end
```



Implémentation d'un menu interactif pour
l'amélioration du mini-projet :

Menu et boucle principale du mini-projet :

Le programme propose un **menu interactif** permettant à l'utilisateur de choisir différentes opérations sur les images. La boucle principale est une structure `while true` qui continue de s'exécuter

jusqu'à ce que l'utilisateur décide de quitter (option 14).

1. Fonction menu

- La fonction menu affiche la liste des options disponibles :
 1. Ouvrir une image
 2. Créer une image noire et blanche
 3. Construire le négatif d'une image
 4. Ouvrir l'image A définie dans l'énoncé
 5. Inverser une image
 6. Symétrie verticale de l'image
 7. Poser verticalement deux images
 8. Poser horizontalement deux images
 9. Afficher les valeurs d'une matrice RGB prédéfinie
 10. Initialiser et afficher une image RGB aléatoire
 11. Symétrie horizontale (haut-bas)
 12. Symétrie verticale (gauche-droite)

- 13. Convertir une image RGB en niveaux de gris
 - 14. Quitter
- La fonction récupère le choix de l'utilisateur avec input.
-

2. Boucle principale while true

- **Option 14 : Quitter**

- Affiche "Au revoir!" et termine la boucle avec break.

- **Options 1 à 13 : Traitements d'images**

- Selon le choix de l'utilisateur, la boucle exécute les fonctions correspondantes :

- 1. lectureImage et afficher_image pour ouvrir une image.

- 2. image_noir_blanche pour créer un damier noir et blanc.

- 3. negatif pour calculer le négatif d'une image.

- 4. Ouvrir l'image A définie dans l'énoncé.

5. inverser pour inverser l'image.
6. flipH pour effectuer une symétrie verticale.
7. poserV pour superposer verticalement deux images.
8. poserH pour juxtaposer horizontalement deux images.
9. Affichage des valeurs de la matrice RGB prédéfinie.
10. initImageRGB pour créer et afficher une image couleur aléatoire.
11. symetrie_horizontale pour la symétrie haut-bas.
12. symetrie_verticale pour la symétrie gauche-droite.
13. Conversion RGB → niveaux de gris avec grayscale et affichage.

• Gestion des erreurs

- Les try-catch permettent de gérer les **chemins d'accès invalides** ou les erreurs lors de l'importation des images.

- Des messages d'erreur clairs sont affichés pour guider l'utilisateur.
 - **Affichage**
 - Toutes les images traitées sont affichées avec la fonction `afficher_image`.
 - Les résultats sont visibles directement dans une fenêtre MATLAB.
-

3. Principe général

- Le programme est **modulaire** : chaque option correspond à une fonction distincte.
- L'utilisateur peut **tester et visualiser différentes opérations** sans relancer le programme.
- Le menu interactif permet une **navigation simple et intuitive**, adaptée au mini-projet de traitement d'images.
- Les fonctions sont combinées avec la **boucle principale** pour créer un flux logique : ouverture → traitement → affichage.

```

%% Boucle principale
while true
    choix = menu();

    if choix == 14
        fprintf("Au revoir!\n");
        break;
    elseif choix == 1
        try
            m = input("Entrez le chemin d'accès de l'image sans guillemets: ", 's');
            img = lectureImage(m);
            afficher_image(img);
        catch ME
            fprintf("Image non trouvée, veuillez écrire un chemin d'accès valide\n");
        end
    elseif choix == 2
        h = input("Entrez la hauteur de l'image: ");
        l = input("Entrez la largeur de l'image: ");
        img = image_noir_blanche(h, l);
        afficher_image(img);
    elseif choix == 3
        chemin = input("Entrez le chemin d'accès de l'image A sans guillemets: ", 's');
        img = lectureImage(chemin);
        neg = negatif(img);
        afficher_image(neg);
    elseif choix == 4
        chemin = input("Entrez le chemin d'accès de l'image sans guillemets: ", 's');
        imgA = lectureImage(chemin);
        afficher_image(imgA);
    elseif choix == 5
        chemin = input("Entrez le chemin d'accès de l'image sans guillemets: ", 's');
        img = lectureImage(chemin);
        inv = inverser(img);
        afficher_image(inv);
    end
end

```

Command Window

```

ohcine\Documents\S3\INFO\MATLAB\MATLAB.m'
1. Ouvrir une image
2. Créer une image noire et blanche
3. Construire le négatif de l'image donnée
4. Ouvrir l'image A
5. Inverser une image
6. Symétrie d'axe vertical passant par le milieu de l'image
7. Poser verticalement deux images
8. Poser horizontalement deux images
9. L'affichage des valeurs du matrice M de l'images RGB
10. Initialiser et renvoyer une image RGB
11. Symétrie horizontale d'une image
12. Symétrie verticale d'une image
13. Convertir une image RGB en niveaux de gris
14. Quitter

```

• Conclusion:

- Ce mini-projet avait pour objectif de mettre en pratique les notions fondamentales du **traitement d'images numériques** à l'aide de MATLAB. À travers la conception d'un programme interactif basé sur un menu, plusieurs opérations essentielles ont été implémentées et testées, telles que l'ouverture d'images, la conversion en niveaux de gris, l'inversion, le calcul du négatif, les symétries, ainsi que la manipulation d'images RGB.

- L'approche modulaire adoptée, basée sur des fonctions indépendantes, a permis d'obtenir un code clair, structuré et facilement réutilisable. L'utilisation d'une boucle principale avec un menu interactif rend le programme simple à utiliser et adaptable à différents scénarios, tout en facilitant la visualisation immédiate des résultats grâce à l'affichage graphique.
- Ce mini-projet a permis de mieux comprendre la **représentation matricielle des images**, le rôle des canaux de couleur, ainsi que l'impact des opérations mathématiques sur l'apparence visuelle d'une image. Il constitue une base solide pour des applications plus avancées, telles que la détection de contours, le filtrage d'images ou encore l'analyse d'images médicales et industrielles.
- En perspective, ce travail pourrait être enrichi par l'ajout d'une interface graphique (GUI), l'intégration de filtres avancés ou l'automatisation de certains traitements, afin d'élargir les fonctionnalités du programme et d'approfondir les compétences en traitement d'images.